Téma 2: Základy algoritmizace

Teoretický základ

Programování je z 80 % přemýšlení a z 20 % psaní kódu. Algoritmus je přesný návod k vyřešení problému.

- **Vlastnosti správného algoritmu:**
  - *Konečnost:* Každý algoritmus musí po určitém počtu kroků skončit. Nesmí běžet do nekonečna.
  - *Determinovanost:* Za stejných výchozích podmínek musí dát vždy stejný výsledek. Krok nesmí být dvojznačný.
  - *Rezultativnost:* Musí vést k určitému výsledku (výstupu).
- **Zápis algoritmu:** Používáme vývojové diagramy (grafické tvary) nebo pseudokód (strukturovaný text podobný mluvenému jazyku).

Základní koncept k zamyšlení: Návrh algoritmu

Jak by vypadal algoritmus pro nalezení největšího čísla v balíčku náhodně zamíchaných karet? Zapsáno v pseudokódu:
1. Vezmu první kartu, její hodnotu si uložím jako "zatím největší".
2. Vezmu další kartu. Pokud je její hodnota větší než "zatím největší", přepíšu "zatím největší" touto novou hodnotou.
3. Opakuji krok 2, dokud nedojdou karty v balíčku.
4. Oznámím hodnotu, kterou mám uloženou jako "zatím největší".

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Napište Python kód, který ze zadaného pole (seznamu) čísel `[4, 2, 9, 1, 5]` najde největší číslo bez použití vestavěné funkce `max()`. Kód musí fungovat nezávisle na obsahu seznamu.

Řešení úkolu 1

```
# Nalezení maxima
cisla = [4, 2, 9, 1, 5]

# Předpokládáme, že první prvek je na začátku největší
nejvetsi = cisla[0]

for cislo in cisla:
    if cislo > nejvetsi:
        nejvetsi = cislo  # Našli jsme větší, aktualizujeme paměť

print(f"Největší číslo je: {nejvetsi}")
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Napište algoritmus, který zjistí, zda zadané slovo je palindrom (čte se zleva i zprava stejně, např. "kajak"). Pokud ano, vypište potvrzení, v opačném případě vypište, že o palindrom nejde.

Řešení úkolu 2

```
# Detekce palindromu
slovo = "kajak"

# K obrácení řetězce v Pythonu se často využívá tzv. slicing (krájení).
# Zápis [::-1] znamená 'od začátku do konce, ale krokem mínus jedna'.
obracene_slovo = slovo[::-1]

if slovo == obracene_slovo:
    print(f"Slovo '{slovo}' je palindrom.")
else:
    print(f"Slovo '{slovo}' není palindrom.")
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Klasický algoritmický problém: FizzBuzz. Vypište čísla od 1 do 100. Pokud je číslo dělitelné 3, vypište místo něj "Fizz". Pokud 5, vypište "Buzz". Pokud 3 i 5 zároveň, vypište "FizzBuzz".

Řešení úkolu 3

```
# FizzBuzz - Důležité je pořadí podmínek!
for i in range(1, 101):

    # Nejpřísnější podmínku (dělitelnost oběma čísly) musíme kontrolovat jako první,
    # jinak by se vyhodnotilo 'Fizz' a program by se k FizzBuzz nedostal.
    if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: Napište vlastní algoritmus (kód v Pythonu), který sečte všechny hodnoty v poli (seznamu) `[10, 20, 30, 40]`. Nesmíte využít zabudovanou funkci `sum()`, musíte logiku vymyslet pomocí for cyklu.

Řešení úkolu 4

```
cisla = [10, 20, 30, 40]

# Paměťová proměnná (tzv. akumulátor), do které budeme sbírat výsledek
celkovy_soucet = 0

for cislo in cisla:
    # V každém kroku cyklu přičteme aktuální hodnotu k dosavadnímu součtu
    celkovy_soucet += cislo

print(f"Součet čísel v poli je: {celkovy_soucet}")
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: (Lineární vyhledávání) Máte seznam `["jablko", "banan", "hruska", "meloun"]`. Napište algoritmus, který najde, na jakém indexu (na kolikáté pozici) se nachází "hruska". Využijte cyklus počítající od 0 výše. Nepoužívejte vestavěnou metodu `.index()`.

Řešení úkolu 5

```
ovoce = ["jablko", "banan", "hruska", "meloun"]
hledane = "hruska"

for i in range(len(ovoce)):
    # Porovnáme hledaný výraz s prvkem na aktuální pozici (i)
    if ovoce[i] == hledane:
        print(f"Položka nalezena na indexu: {i}")
        break # Jakmile najdeme, ukončíme hledání
else:
    # Tento else se váže k cyklu, provede se jen pokud se neaktivoval break
    print("Položka se v poli nenachází.")
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: Seřazení hodnot (Bubble Sort). Máte nesetříděné pole `[5, 2, 8, 1, 9]`. Napište algoritmus pro jeho seřazení od nejmenšího po největší. Krok za krokem porovnávejte vždy dva sousední prvky; pokud je první větší než druhý, prohoďte je.

Řešení úkolu 6

```
cisla = [5, 2, 8, 1, 9]
n = len(cisla)

# Vnější cyklus zajišťuje průchod celým polem potřebně krát
for i in range(n):
    # Vnitřní cyklus porovnává sousedy.
    # S každým oběhem největší číslo "probublá" na konec,
    # proto vpravo už nemusíme kontrolovat (n - i - 1).
    for j in range(0, n - i - 1):
        if cisla[j] > cisla[j + 1]:
            # Prohození (Swap) v Pythonu lze elegantně na jednom řádku:
            cisla[j], cisla[j + 1] = cisla[j + 1], cisla[j]

print(f"Seřazené pole: {cisla}")
```